SAVANNA SENTINEL

Sigill

Coding Standards

Last Revision: July 13 2026

David van Rooijen

Dandré Nel

Daniel Cohen

Stephan Kritzinger

Michael Koch

TABLE OF CONTENTS

File Structure

NOTE: Emdash was copypasted since it makes a better horizontal line than the dash on my keyboard.

```
Savanna-Sentinel/
|
|——backend/
|   |——init-db/          #Contains files to seed and initialise the
database
|   |——tests/            #Contains the test files for the backend
|      |——integration/   #Contains the integration test files
|      └——unit/          #Contains the unit test files
|   |——app/
|      |——api/           #Contains the files used by various aspects of
fastAPI
|         └——v1/         #Contains route information
|      |——core/          #Contains helper functions and some config
files
|      |——models/        #Contains the database table structure
|      |——repositories/  #Contains files to interface with the database
and retrieve data from the database
|      |——schemas/       #Contains DTO's and ViewModels to ensure data
integrity
|      |——services/      #Contains main API functionality which router
executes
|      └——workers/       #Contains celery/redis workers
|   |——Dockerfile
|   |——requirements.txt  #Contains Package information
|   └——.env              #User-generated file, contains keys and
secrets.
|——frontend/
|   |——public/
|      └——icons/         #Contains static content used throughout the
dashboard
|   |——src/              #Contains most frontend functionality
|      |——components/
|         |——ui/         #Contains general re-usable components that are
used on multiple pages
```

```
|         └──{page_name}/#Contains reusable components specific to that
page
|      |──hooks/          #Contains interceptors for every request when
activated for the page.
|      |──lib/            #Contains helper functions
|      |──offline/        #Contains offline functionality for PWA
|      |──pages/          #Contains the raw page code for each route
|      |──services/       #Contains API call functions and helper
functions
|      |──store/          #Contains persistent data for the session
|      |──styles/         #Contains scss files
|      |──tests/          #Contains frontend testing files
|         |──mocks/       #Contains mocks for unit tests
|         └──{file.test.tsx}
|      |──App.tsx         #The containing page of the SPA and also
routing
|      └──main.tsx        #Wrapper for the App page
|   |──Dockerfile
|   └──.env               #Frontend environment configuration
|──docs/
|   |──architecture/
|   |──design/
|   |──drafts/            #Contains .md drafts of docs, before formatting
has occurred and transformed to pdf
|   |──non-functional/
|   |──requirements/
|   └──research/          #Contains research of AI methods to be used
└──docker-compose.yml
```

Naming conventions (Backend)

- File - Snake Case (e.g. user_service.py)
- Methods - Snake Case
- Variables - Snake Case
- Classes & Interfaces - Pascal Case
- Folders - Attempt 1 word folder names, otherwise kebab case (e.g. init-db)
- Constants - Upper Snake Case (e.g. CONSTANT_VARIABLE)

Naming conventions (Frontend)

- Files w/ Components - Pascal Case (e.g. DataPreview.tsx)
- Files w/ Helpers - Camel Case
- Methods - Camel Case (Matching React Syntax)
- Variables - Camel Case
- Components - Pascal Case
- Folders - Attempt 1 word folder names, otherwise kebab case
- Constant Globals - UPPER SNAKE CASE (Otherwise follow Camel Case like a normal variable)

Coding Style

Adapted from PEP 8, although with modifications.

- Never exceed 80 characters on a line
    - Make a new line at a logical location if threatening to exceed the limit.
- Either define all formal parameters on the same line, or each one on a new line with extra indentation from the body to ensure readability. E.g.

  Correct

```
async def admin_delete_user(
       user_id: str,
       db: AsyncSession = Depends(get_db),
       current_admin: User = Depends(require_admin)
       ):
   repo = UserRepository(db)
   service = UserService(repo)
```

```
async def change_role(self, user_id: str, new_role: str):
   result = await self.repo.update_role(user_id, new_role)
   return result
```

- Separate logical sections inside a method with a single blank line.
- Separate method definitions with a single blank line
    - This can be ignored if methods are related and minimal, such as getters or setters.
- Separate logical groups of methods with 2 blank lines.
- All boolean variables must contain a prefix that phrases the variable as a question. (e.g. is_correct/isCorrect)
- Imports must only be at the very top of the file.

- Include all method imports from a single package onto a single line or encapsulate in brackets
- All Globals must be defined under imports
- All strings must be encapsulated with double quotes("), using ` for template strings when necessary. Single quotes(') should be avoided to list a string.
    - Triple double quotes(""") must be used for docstrings
- Arrays or similar data structures must include a trailing comma, for easy expansion later. E.g.

```
example_array = {
   "data_item_1": "test",
   "data_item_2": "test2", <-- trailing comma
}
```

- All arrays must use the above syntax, with 1 item being included per line
- All functions must define a return type, with typed Python or TypeScript interfaces.
    - The use of the any type should be avoided.
- Strict Equality should be used when possible (=== over ==)
- The use of the var word in the frontend is forbidden, use modern JS syntax such as let or const, preferring const unless the variable is designed to be changing.
- Binary operators must be surrounded by 2 empty spaces (1 + 2)
- Attempt to keep all variable declarations at the top of their respective block scope.
- Comments must be placed above the method/line they are explaining
    - Inline comments are forbidden.
- Arrow syntax (=>) is preferred for Javascript function definitions.
- A frontend file that contains Markdown language (\<tag>) must use the .tsx file extension. Else .ts must be used.
- All props must be destructured inside their component definition

Correct

```
const ExampleComp = ({itemId, isValid}) => {
   ...
}
```

Incorrect

```
const ExampleComp = (props: ExampleCompProps) => {
   const itemId: string = props.itemId
}
```

Linters and configuration

Eslint

```
export default defineConfig([
   globalIgnores(["dist", "coverage", "web-build"]),
   {
       ignores: ["src/components/ui", "vite.config.ts"],
   },
   {
       files: ["**/*.{ts,tsx}"],
       plugins: {
           "@stylistic": stylistic,
           react: reactPlugin,
       },
       extends: [
           js.configs.recommended,
           ...tseslint.configs.recommended,
           reactHooks.configs.flat.recommended,
           reactRefresh.configs.vite,
       ],
       languageOptions: {
           globals: globals.browser,
           parser: tseslint.parser,
           parserOptions: {
               project: ["./tsconfig.app.json"],
               tsconfigRootDir: import.meta.dirname,
               ecmaFeatures: {
                   jsx: true,
               },
           },
       },
       settings: {
           react: {
               version: "19.2",
           },
       },
       rules: {
```

```
            //Enforce strict equality
            eqeqeq: ["error", "always"],
            "no-var": "error",
            "prefer-const": "error",
            //typescript rules
            //If overloads are not consecutive, throw an error
            "@typescript-eslint/adjacent-overload-signatures": "error",
            "@typescript-eslint/array-type": ["error", { default: "array"
}],
            //Enforce variable names according to standard
            "@typescript-eslint/naming-convention": [
                "error",
                {
                    selector: "variable",
                    types: ["boolean"],
                    //Eslint will strip out the is has should words when
parsing due to the prefix rule, so Pascal is used to avoid the error.
                    //Camel should be used in all cases
                    format: ["PascalCase"],
                    prefix: ["is", "has", "should", "can", "show"],
                },
                {
                    selector: "variable",
                    types: ["function"],
                    format: ["camelCase", "PascalCase"],
                },
                {
                    selector: ["function", "method"],
                    format: ["camelCase", "PascalCase"],
                    leadingUnderscore: "allow",
                },
                {
                    selector: "variable",
                    format: ["camelCase"],
                    leadingUnderscore: "allow",
                },
                {
                    selector: ["typeLike"],
                    format: ["PascalCase"],
                },
```

```
                {
                    selector: "variable",
                    modifiers: ["global"],
                    format: ["UPPER_CASE", "camelCase"],
                },
            ],
            "react/destructuring-assignment": ["error", "always"],
            "@stylistic/line-comment-position": [
                "error",
                { position: "above" },
            ],
        },
    },
]);
```

Prettier

```
{
    "printWidth": 80,
    "tabWidth": 4,
    "useTabs": false,
    "semi": true,
    "singleQuote": false,
    "jsxSingleQuote": false,
    "quoteProps": "as-needed",
    "trailingComma": "all",
    "bracketSpacing": true,
    "bracketSameLine": false,
    "objectWrap": "preserve",
    "arrowParens": "always"
}
```

Ruff

```
# https://docs.astral.sh/ruff/rules/
[tool.ruff]
```

```
target-version = "py311"
line-length = 80
[tool.ruff.lint]
# E - PEP8
# W - PEP8
# F - Find basic errors and undefined variables
# I - Sort import statements and blocks
# N - Enforce Case constraints
# D - Docstring rules
# COM - Enforce trailing commas
# Q - Enforce string quote rules
# TCH - Enforce type checking
select = ["E", "W", "F", "I", "N", "D", "COM", "Q", "TCH"]
ignore = [
   "D100",
   "D104",
   "D102",
   "D103",
   "D107",
   "D101",
]
[tool.ruff.lint.pydocstyle]
convention = "pep257"
[tool.ruff.lint.extend-per-file-ignores]
# Ignore missing docstringsin test files
"tests/**/*.py" = ["D103", "D102", "D107", "D101"]
[tool.ruff.lint.flake8-quotes]
inline-quotes = "double"
multiline-quotes = "double"
docstring-quotes = "double"
[tool.ruff.lint.pep8-naming]
# Classes & Interfaces - PascalCase
# Methods & Variables - snake_case
# Constants - UPPER_SNAKE_CASE
classmethod-decorators = ["classmethod"]
[tool.ruff.format]
quote-style = "double"
indent-style = "space"
skip-magic-trailing-comma = false
docstring-code-format = true
```

Before Commits

- Ensure all testing and linting is run locally before committing, to improve CI success rate.
- Ensure PR have a clear description, and also mention parts that might need to be investigated further.

Github/Git

- All commits and pushes made directly to github must be made on a branch of dev
    - Dev and main are not allowed to be pushed to directly, code can only enter through PR's through review of a separate developer.
- All branch names must follow the following convention type/name
    - Type consists of
        - Fix
        - Task
        - Feature
    - Name is a short description of what is being done in snake case.
- All CI must succeed before a PR can be accepted
    - In rare circumstances, such as SonarQube failing, the team leader may make an executive decision to ignore it after careful review
    - Failing test cases cannot be ignored nor bypassed.
- All commits must have a short but descriptive message as to the commits contents.

Changelog

V2.1

[13 July 2026]

- Updated linters to their current configuration
- Minor improvements